Computing for Analytical Work

Session 1 — Where am I?

Block 1 — Files, folders, paths, and working directories

This block in one sentence

Programs do not use your intentions. They use paths from a current working directory.

The whole block is the gap between two statements:

- "The file exists."
- "The program is looking in the right place."

Those are different claims. Today you learn to tell them apart.

Agenda

1. Files, folders, and trees
2. Absolute paths vs. relative paths
3. The current working directory and the project root
4. "Where am I?" — the terminal answer, then the Python answer
5. Why notebooks lose files
6. Git thread: what is a repo, what does `git status` say
7. AI thread: a good prompt vs. "fix this"
8. Stretch, then Lab 1

Files and folders are a tree

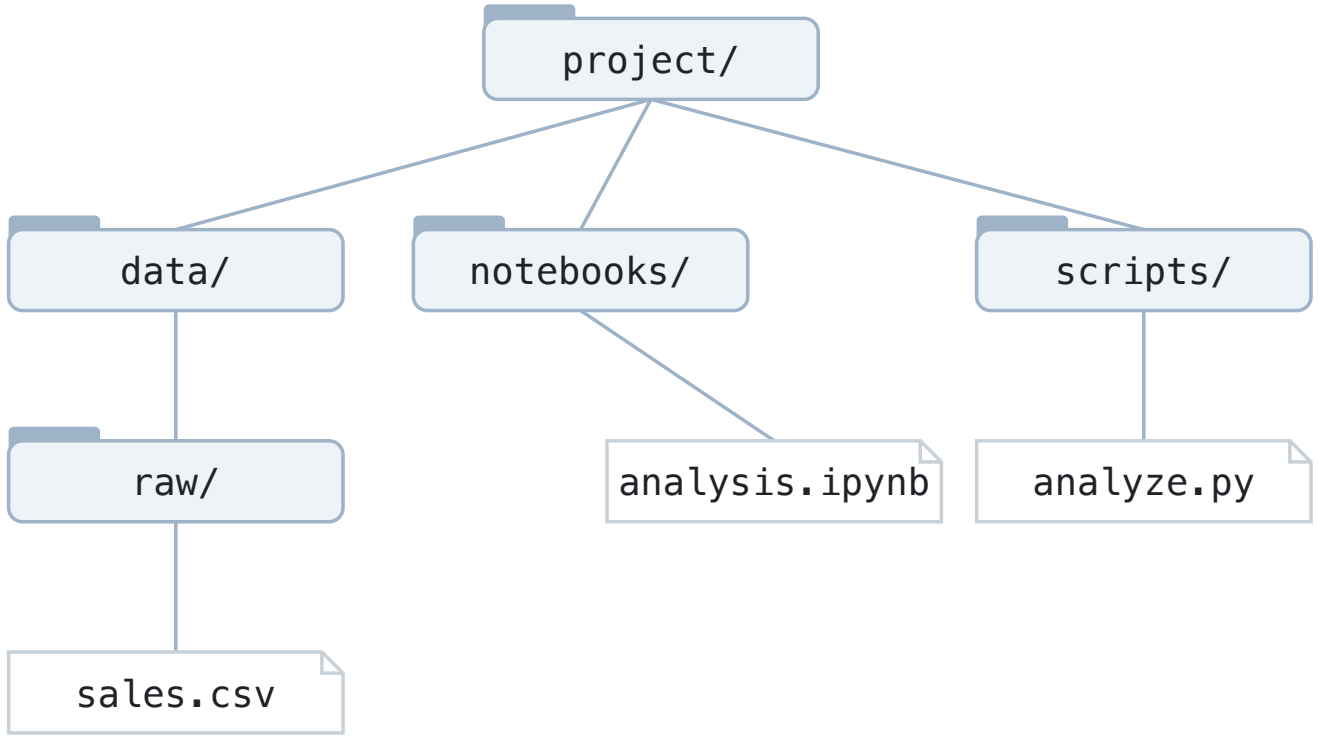
- A **file** holds content: bytes on disk.
- A **folder** (directory) holds files and other folders.
- Every folder has exactly one parent, except the top.
- That makes the whole disk a **tree** with one root.
- A path is just directions through the tree.

Nothing about this is Python-specific. Your operating system was doing this before Python started.

The tree we will use all session

```
project/
  data/
    raw/
      sales.csv
  notebooks/
    analysis.ipynb
  scripts/
    analyze.py
```

Three folders under the project. One data file, one notebook, one script. Every example today lives here.



Absolute paths

An absolute path starts at the root of the tree and spells out every step.

```
/Users/anna/project/data/raw/sales.csv        macOS
C:/Users/anna/project/data/raw/sales.csv      Windows (Git Bash shows /c/Users/...)
```

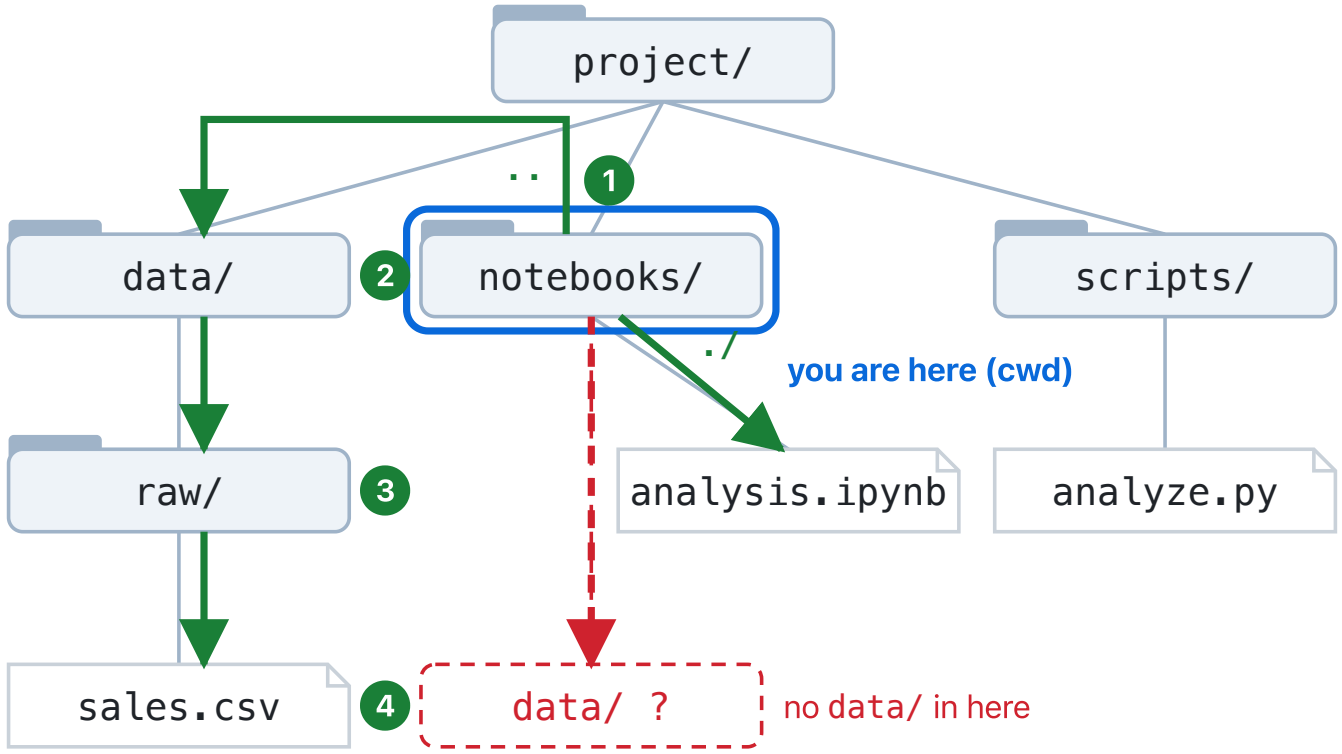
- Unambiguous: it means the same thing no matter where you are.
- Fragile: it is true only on **this** machine, for **this** user.

Relative paths

A relative path starts from *somewhere* and gives directions from there.

```
data/raw/sales.csv
../data/raw/sales.csv
./analysis.ipynb
```

- `..` means "the parent folder". `.` means "this folder".
- Portable across machines. But only meaningful once you know the starting point.



✗	`data/raw/sales.csv`	looks for notebooks/data/ — nothing there
✓	`../data/raw/sales.csv`	up to project/ first, then down
✓	`./analysis.ipynb`	this folder, then the file

Relative to what? The current working directory

Every running program has a **current working directory** (cwd): the folder it is standing in right now.

- Relative paths are resolved **from the cwd**. Not from the file the code is in.
- The cwd is set when the program starts, by whoever started it.
- It is a property of the *process*, not of your code or your intentions.

`data/raw/sales.csv` + cwd `project/` → found.
`data/raw/sales.csv` + cwd `project/notebooks/` → not found.

The project root

The **project root** is the top folder of one project: where the README, `pyproject.toml`, and `.git` live.

- Convention in this course: **paths in code are relative to the project root.**
- So the program must be *started* from the project root.
- If it was not, either move the program or fix the path. Never move the data so a wrong path starts working — decide first whether the path or the layout is wrong.

Every lab README tells you: "run this from the project folder." That sentence is the cwd contract.

"Where am I?"

The terminal answer, then the Python answer

`pwd` and `ls`

```
pwd     # print working directory — where am I?
ls      # list — what is here?
```

```
$ pwd
/Users/anna/project
$ ls
README.md  data  notebooks  pyproject.toml  scripts
```

`pwd` answers the question. `ls` shows what the answer contains. Run them before you trust anything.

`cd` — and then ask again

```
cd notebooks && pwd
```

```
/Users/anna/project/notebooks
```

- `cd` changes the shell's working directory. Nothing else moves.
- After every `cd`, the answer to "where am I?" is different. Every relative path is now resolved from the new place.
- `cd ..` goes up one level. `cd` with no argument goes home.

Predict before you look

You are standing in `project/notebooks/`. You type:

```
ls data/raw/sales.csv
```

Write down two things, before the next slide:

1. The **full path** the shell will actually try.
2. Whether that path **exists** in the tree from earlier.

Thirty seconds. No typing.

The same path, two answers

```
cd /Users/anna/project/notebooks
ls data/raw/sales.csv        # fails — I am in notebooks/, not the root
ls ../data/raw/sales.csv     # works
```

```
anna@laptop project $ pwd
/Users/anna/project
anna@laptop project $ ls
data            notebooks       pyproject.toml  README.md       scripts
anna@laptop project $ cd notebooks
anna@laptop notebooks $ pwd
/Users/anna/project/notebooks
anna@laptop notebooks $ ls data/raw/sales.csv
ls: data/raw/sales.csv: No such file or directory
anna@laptop notebooks $ ls ../data/raw/sales.csv
../data/raw/sales.csv
anna@laptop notebooks $
```

The file did not move. My position did. `No such file` means "nothing at that path **from here**." It says nothing about whether the file exists elsewhere; that is your next question, and `ls` answers it.

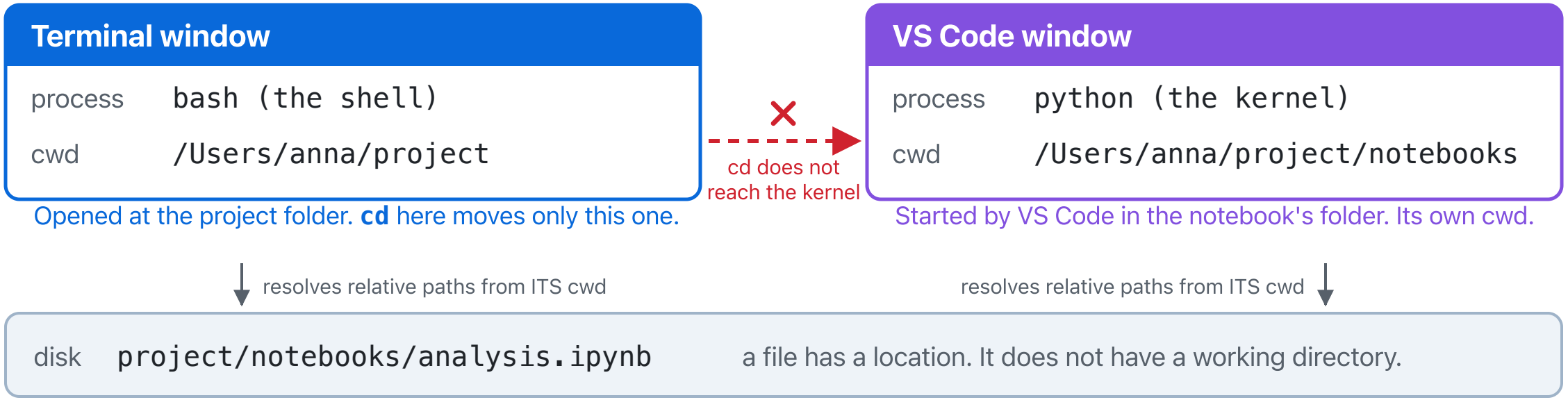
The Python answer to the same question

```
import os
from pathlib import Path

os.getcwd()                                  # '/Users/anna/project/notebooks'
Path.cwd()                                   # the pathlib equivalent
Path("data/raw/sales.csv").exists()          # False from notebooks/
Path("../data/raw/sales.csv").exists()       # True
```

`os.getcwd()` is Python's `pwd`. Start Python **from this shell** and it inherits the shell's cwd — same answer. A notebook kernel is a **different process**: ask *it*, not your terminal.

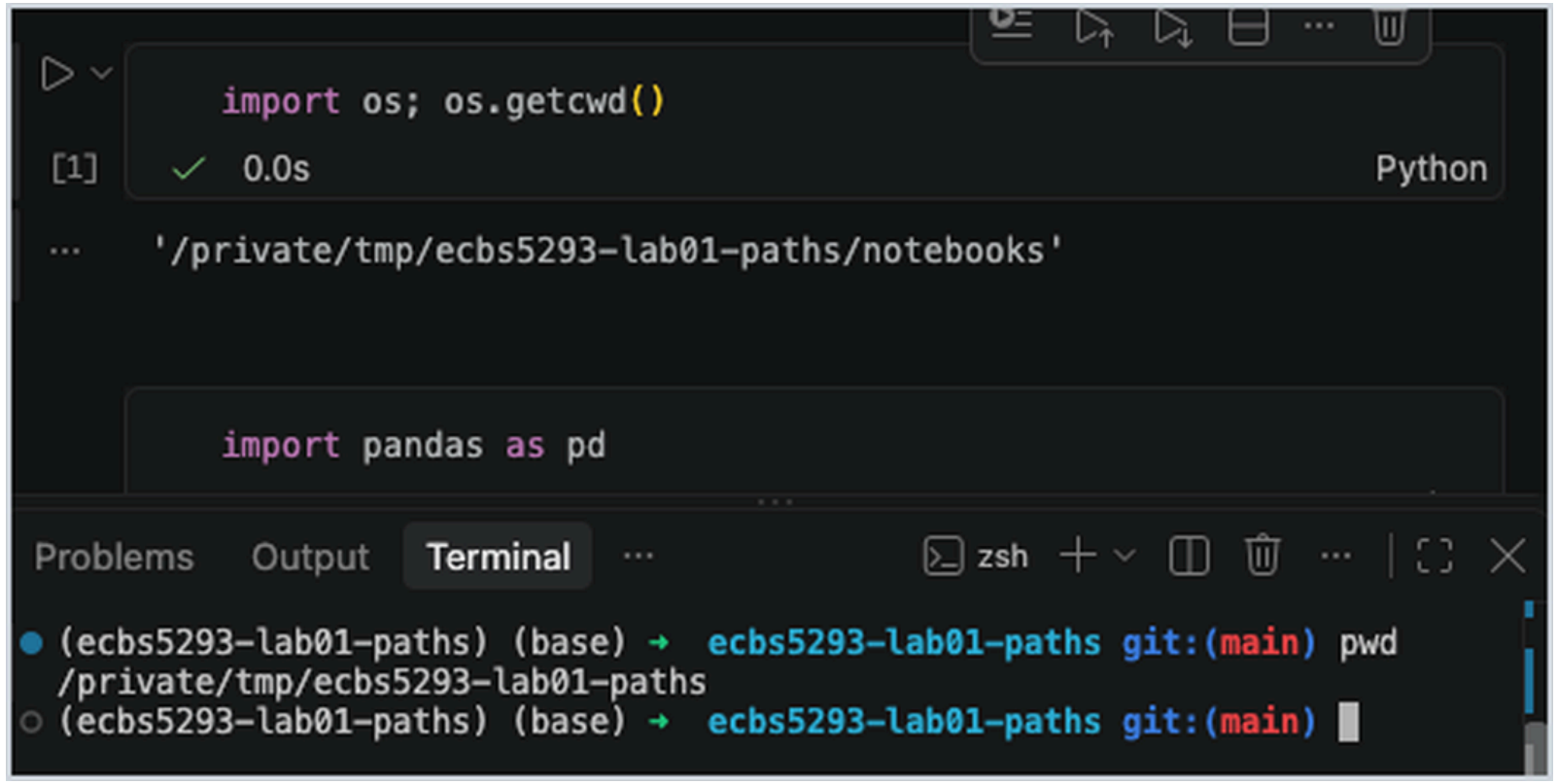
Why notebooks lose files



Two processes. Two working directories. Ask the one that runs your code.

First thing to check when a relative path fails in a notebook: `os.getcwd()`. Not the file browser. The kernel. `cd` in your terminal does **not** move the kernel.

Two answers, one screen



The cell asked the **kernel**; the terminal asked the **shell**. Two processes, two working directories, neither wrong.

"The file exists" vs. "the program is looking in the right place"

Claim	How you check it
The file exists	`ls`, the file browser, `Path(...).exists()` with an absolute path
The program is looking in the right place	`pwd` / `os.getcwd()`, then resolve the relative path *from there*

A `FileNotFoundError` does not tell you which row failed. Check both: row one with `ls`, row two with the cwd. *Insisting* on row one is not the same as *checking* it.

Local vs. remote, conceptually

- **Local**: the tree on the machine in front of you. `pwd` and `os.getcwd()` describe it.
- **Remote**: a different machine with its own tree — a server, a cloud notebook, a colleague's laptop.
- A path is only meaningful **on the machine where the process runs.**
- `/Users/anna/project` on your Mac does not exist on the server. On a cloud notebook, "your files" are on *their* disk.

When someone says "it works on my machine," they are usually describing a different tree.

Three honest fixes, one dishonest one

When `data/raw/sales.csv` fails from `notebooks/`:

1. **Move the program's position**: start it from the project root. Best when the README promises that.
2. **Fix the path** relative to where the program runs: `../data/raw/sales.csv`. Works from one cwd only.
3. **Anchor to the project root**, so it works from both. The recommended fix for notebooks:

```
from pathlib import Path
ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
pd.read_csv(ROOT / "data/raw/sales.csv")
```

4. ~~Copy the CSV into `notebooks/` so the wrong path works.~~ Two files, one stale, nobody knows which.

Fix the *diagnosis*, not the *symptom*. Then verify by loading the data.

Git thread — what is a repo?

- A **repository** is a project folder that Git is watching. The evidence is a hidden `.git/` folder at the project root.
- Git records **snapshots** of the files over time, so you can see what changed and go back.
- Every lab and homework in this course is a repo you clone.
- You do not need to know Git to use `git status`. You need it to know one thing: *what is the state of my project right now?*

Git thread — `git status`

```
git status
```

```
On branch main
Changes not staged for commit:
        modified:   notebooks/analysis.ipynb
Untracked files:
        notebooks/sales.csv
```

- **modified**: a tracked file you changed. **untracked**: a file Git has never seen. Nothing listed: unchanged.
- Run it from the project root. Run it *before* you start and *after* you fix. The diff between those two moments is your work.

AI thread — a prompt with evidence in it

```
I am working in a Python project. My notebook is in notebooks/analysis.ipynb.
The data file is in data/raw/sales.csv.
When I run pd.read_csv("data/raw/sales.csv"), I get FileNotFoundError.
I ran os.getcwd() in the notebook and it returned /Users/me/project/notebooks.

What is likely happening? What should I inspect before changing the code?
```

Four pieces of evidence: the error, the expected path, the real location, the actual cwd. Then a question about *causes*, not a demand for a fix.

AI thread — "fix this" vs. the filesystem

```
Fix this code.
```

- No cwd, no tree, no error text. The assistant will guess, confidently.
- Even a good answer is a **hypothesis**. Verify it against reality: `pwd`, `ls`, `os.getcwd()`, `Path(...).exists()`.
- **AI can explain the symptom. The filesystem is the source of truth.**
- The habit: exact symptom → context (tree, cwd, command) → ask for causes and checks → one small change → verify → explain it in your own words.

The diagnosis note and the checkoff

Every lab ends with a short note, five questions:

1. What was the symptom?
2. What was the actual cause?
3. What evidence showed that? Paste the raw output: cwd, the traceback lines.
4. What did you change?
5. How did you verify it worked?

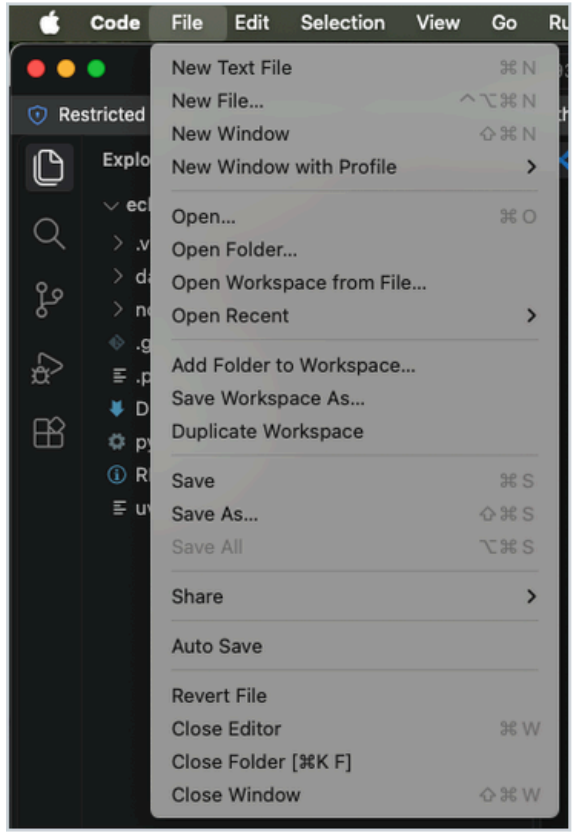
Then a TA spends 60–90 seconds at your desk. No notes, no AI. Symptom, cause, change, verification — in your own words.

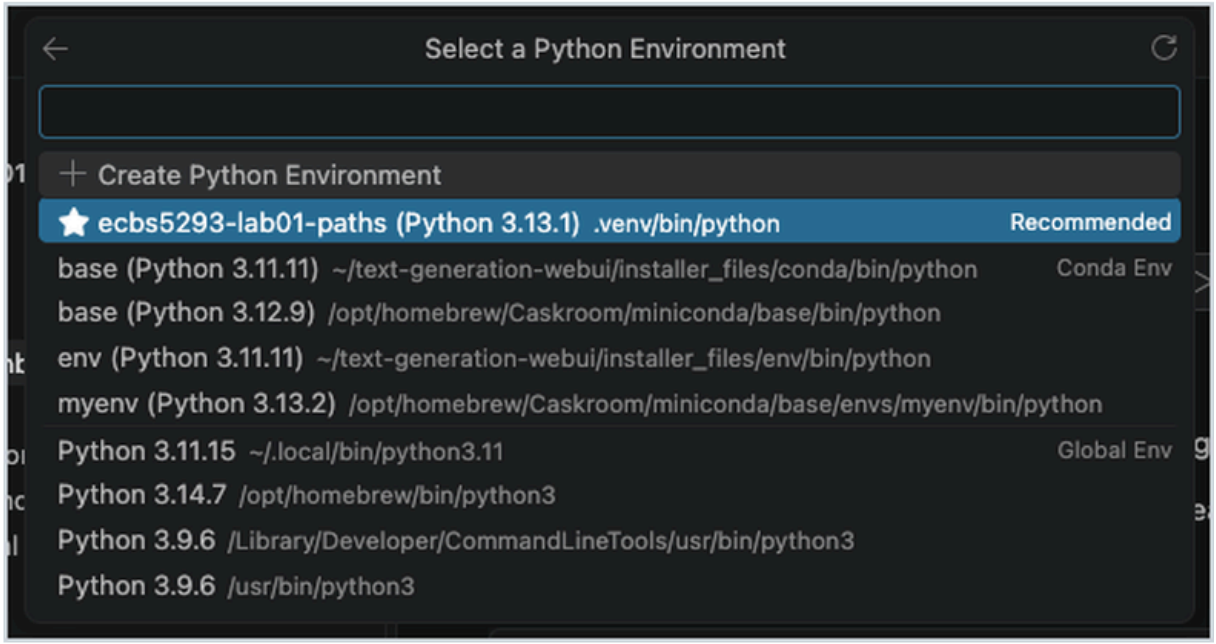
5-minute stand-and-stretch

Stand up. Leave the laptop. Water, window, hallway.

Back in five minutes, then 45 minutes of lab.

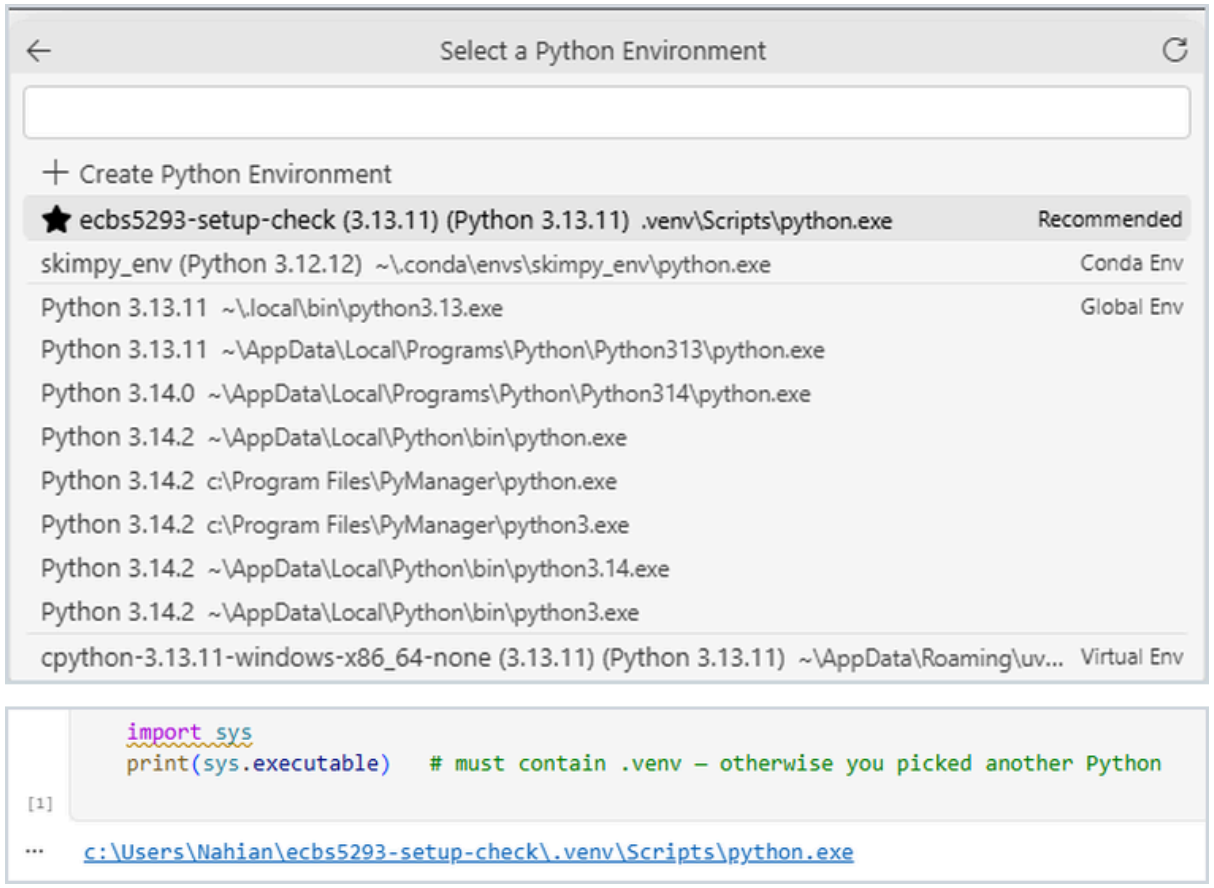
Before the lab: open the folder, pick the kernel





File → Open Folder…: the project folder, not the notebook file. Then **Select Kernel**: the entry that lives in this project's `.venv/`, marked *Recommended*. Anything called base, conda, or a bare version number is the wrong Python.

The same picker on Windows



`.venv\Scripts\python.exe`, marked *Recommended*, above eleven other Pythons on one laptop. Below it: the setup notebook's first cell, run on that kernel.

Lab 1 — The missing file that is not missing

`notebooks/analysis.ipynb` runs `pd.read_csv("raw/sales.csv")` and gets `FileNotFoundError` — from *any* working directory. Inspect the tree and the cwd. Find the real file. Fix the path. Verify by loading the data. Write the diagnosis note. Get checked off.

Do these exactly — Session 2 explains what they mean:

1. Terminal, from the project folder: `uv sync`
2. VS Code → **File → Open Folder…** → the project folder
3. Open the notebook; **kernel picker** (top right) → the interpreter in `.venv/`

Recommended fix: the two-line `ROOT` anchor from the lecture. Stretch: a script in `scripts/` that uses the same anchor — run it from the root and from `scripts/`. Which one works, and why?